



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No: 03

Team No: 09

Project Title: Concurrent Web Server for University Resource Portal

Team Members

Roll Number	Student Name	Signature
2520030183	P. Venkata Sri Sai Madhurima	
2520030311	S. Lahari	
2520030070	Revu.Rohith Varma	

1. Abstract

The Concurrent Web Server for University Resource Portal is a Linux-based system designed to provide academic resources such as notices, course materials, examination schedules, and documents to multiple students simultaneously. The project focuses on building a practical user-space server in C using Linux and POSIX mechanisms, with emphasis on concurrency, synchronization, file handling, and resource management.

The existing approach can become inefficient when many students request resources at the same time, leading to delays, increased CPU and I/O usage, and poor responsiveness. The proposed system addresses these limitations through TCP client connections, a worker-thread pool, a shared synchronized request queue, mutexes and semaphores, efficient file access using `mmap()`, activity logging, signal-based control, and resource monitoring. This design provides better concurrent request handling and creates a foundation for performance testing and future scaling.

In conclusion, the proposed server demonstrates important Operating Systems and systems-programming concepts through a functional Linux application. Its modular architecture can be extended with improved caching, configurable worker pools, advanced monitoring, and asynchronous processing, making it suitable for future scalability while maintaining strong academic coverage of OS fundamentals.

2. Problem Statement

Traditional or basic resource-serving systems may experience delays and reduced performance when many students simultaneously request academic files. A system is required that can handle multiple TCP client connections efficiently, manage concurrent requests safely, provide efficient file access, monitor resource usage, and maintain reliable server operation. The proposed project addresses these challenges using Linux/POSIX system-programming mechanisms such as worker threads, synchronization primitives, file I/O, memory mapping, signals, logging, and resource monitoring.

3. Objectives

1. To develop a concurrent web server using C on Linux.
2. To handle multiple TCP client connections simultaneously.
3. To implement a worker-thread pool for efficient request processing.
4. To design a thread-safe shared request queue.
5. To use mutexes and semaphores for synchronization and resource control.
6. To use file descriptors and `mmap()` for efficient file access.
7. To implement signal-based server control and graceful shutdown.
8. To maintain server activity and request logs.
9. To monitor active threads and server resource usage.
10. To measure throughput, response time, and server performance under different workloads.
11. To design the system so that worker capacity and other components can be extended for future scalability.

4. Proposed Methodology

Step 1: Requirement Analysis

Identify the server requirements, supported resources, client requests, concurrency requirements, and performance parameters.

Step 2: Server Socket Creation

Create a TCP socket using Linux socket APIs and configure the server to listen for incoming client connections.

Step 3: Client Connection Handling

Accept multiple client connections and forward incoming requests to a shared request queue.

Step 4: Request Queue Implementation

Create a bounded shared queue for storing client requests safely.

Step 5: Worker Thread Pool

Create multiple POSIX worker threads that retrieve requests from the shared queue and process them concurrently.

Step 6: Synchronization

Use mutexes, condition variables, and semaphores to prevent race conditions and coordinate access to shared resources.

Step 7: File Processing

Use Linux file I/O operations to locate, open, read, and serve requested academic resources.

Step 8: Memory-Mapped File Access

Use `mmap()` for efficient access to suitable static or large files.

Step 9: Logging

Record client requests, file access, errors, completion information, and server activity.

Step 10: Signal-Based Control

Use signals for controlled shutdown, server administration, and configuration-related operations.

Step 11: Resource Monitoring

Monitor active threads and server resource usage using Linux facilities such as `/proc`.

Step 12: Testing and Performance Analysis

Test different numbers of clients, files, worker threads, and request loads. Measure throughput, response time, resource consumption, and overall server behavior.

Step 13: Future Scalability

Design the system modularly so that additional worker threads, caching, improved monitoring, and asynchronous processing can be incorporated later.

Basic Flow of the Proposed System

Client Request → TCP Server → Request Queue → Worker Thread → Synchronization → File Access (`mmap`) → Response → Logging & Monitoring

5. Operating Systems Concepts / Linux APIs Used

S No	OS Concept / Linux API / System Call	Purpose in the Project
1.	<code>socket()</code> , <code>bind()</code> , <code>listen()</code> , <code>accept()</code>	Create and manage TCP server connections
2.	POSIX Threads (<code>pthread_create()</code>)	Create worker threads for concurrent request processing
3.	Mutexes (<code>pthread_mutex_*</code>)	Protect shared queues and data structures
4.	Condition Variables	Coordinate workers waiting for requests
5.	Semaphores (<code>sem_*</code>)	Control access to shared and limited resources
6.	File I/O (<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>)	Access and serve academic resource files
7.	<code>mmap()</code>	Efficient memory-mapped access to files
8.	Signals (<code>signal()</code> , <code>sigaction()</code>)	Server control, shutdown, and event handling
9.	<code>/proc</code> filesystem	Monitor process/thread and system resource information
10.	Dynamic Memory (<code>malloc()</code> , <code>free()</code>)	Manage request, connection, and server data structures
11.	File Descriptors	Manage sockets and opened resource files
12.	<code>fork()</code> / Processes	Optional management or monitoring processes where required
13.	<code>wait()</code> / <code>waitpid()</code>	Synchronize with managed child processes if process-based modules are added
14.	<code>errno</code>	Identify and handle system-call errors
15.	<code>strace</code>	Trace and analyze Linux system calls during testing
16.	<code>gdb</code>	Debug crashes, synchronization issues, and program execution

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030070	Revu.Rohith Varma	Server & Networking: Design and implement TCP socket communication, server socket creation, client connection handling, HTTP request processing, and response handling.
2520030183	P.Venkata Sri Sai Madhurima	Concurrency & Synchronization: Implement POSIX worker threads, shared request queue, mutexes, condition variables/semaphores, and test concurrent client handling.

2520030311	S. Lahari	File Management, Monitoring & Performance: Implement file I/O, <code>mmap()</code> , logging, signal handling, <code>/proc</code> -based resource monitoring, and performance analysis.
------------	-----------	--

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment for the server
C / C++	Implementation of the concurrent web server
TCP/IP Sockets	Establish communication between clients and the server
POSIX Threads (<code>pthread</code>)	Create and manage multiple worker threads
Mutexes	Protect shared resources and prevent race conditions
Condition Variables	Coordinate worker threads and request processing
Semaphores	Synchronize access to the shared request queue
File I/O (<code>open</code> , <code>read</code> , <code>write</code> , <code>close</code>)	Access and serve academic resource files
<code>mmap()</code>	Efficient memory-mapped access to resource files
Signals (<code>sigaction</code> , etc.)	Server control, configuration handling, and graceful shutdown
<code>/proc</code> Filesystem	Monitor CPU, memory, and process/thread information
GCC	Compile the C/C++ source code
Makefile	Automate compilation and project build
GDB	Debug crashes, threads, and synchronization issues
<code>strace</code>	Trace and analyze Linux system calls during testing
Git	Version control and collaborative development
Linux Terminal / Test Scripts	Run the server and perform concurrent client testing

8. Expected Outcome

The proposed system is expected to produce a functional concurrent web server running on Linux that can serve university academic resources such as notices, course materials, examination schedules, PDF files, HTML files, and text documents to multiple clients simultaneously.

The project will demonstrate important Operating Systems and Linux systems-programming concepts, including TCP sockets, POSIX threads, mutexes, semaphores, condition variables, file descriptors, file I/O, memory mapping using `mmap()`, signals, logging, and `/proc`-based resource monitoring.

The system will also provide performance results by testing different numbers of clients and worker threads. The modular design will allow future improvements such as configurable worker pools, caching, asynchronous file processing, enhanced monitoring, and improved scalability. This makes the project suitable for demonstrating both fundamental OS concepts and future scalable system design.

9. GitHub Repository

GitHub Repository Link:

<https://github.com/RevuRohithVarma/Concurrent-Web-Server-for-University-Resource-Portal>

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr. K Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____